CEC Lab 7

Name: Saptanshu Wanjari

Class: A A2

Roll No: 18

```
import tensorflow as tf
import numpy as np
import os
import time

print("TensorFlow Version:", tf.__version__)
```

```
TensorFlow Version: 2.20.0
```

1. Load Dataset

```
(x_train, y_train), (x_test, y_test) = \
    tf.keras.datasets.mnist.load_data()

print("Training data:", x_train.shape)
print("Testing data:", x_test.shape)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-kera
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Training data: (60000, 28, 28)
Testing data: (10000, 28, 28)
```

2. Normalize Dataset

```
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
```

3. Build Model

```
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(28, 28)),
```

```
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax")
])
```

4. Compile Model

```
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

5. Train Original Model

```
model.fit(
    x_train,
    y_train,
    epochs=5,
    validation_split=0.1,
    verbose=1
)
```

```
Epoch 1/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 20s 11ms/step - accuracy: 0.9209 - loss:
Epoch 2/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9638 - loss: 0
Epoch 3/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 10s 5ms/step - accuracy: 0.9751 - loss:
Epoch 4/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 9s 6ms/step - accuracy: 0.9809 - loss: 0
Epoch 5/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.9852 - loss: 0
<keras.src.callbacks.history.History at 0x783745af0ec0>
```

6. Evaluate Original TensorFlow Model

```
loss, original_accuracy = model.evaluate(
    x_test,
    y_test,
    verbose=0
)

print(
```

```
    "\nOriginal TensorFlow Accuracy:",
    original_accuracy * 100,
    "%"
)
```

```
Original TensorFlow Accuracy: 97.39000201225281 %
```

7. Convert Original Model to TFLite

```
converter = tf.lite.TFLiteConverter.from_keras_model(model)
original_tflite = converter.convert()

with open("original_model.tflite", "wb") as f:
    f.write(original_tflite)
```

```
Saved artifact at '/tmp/tmpj9yjpm7w'. The following endpoints are avail

* Endpoint 'serve'
  args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 28, 28), dtype=tf.f
Output Type:
  TensorSpec(shape=(None, 10), dtype=tf.float32, name=None)
Captures:
  132181235188560: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132178791675664: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132178791675280: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132178791676048: TensorSpec(shape=(), dtype=tf.resource, name=None)
```

8. Measure Original Model Size

```
original_size = os.path.getsize("original_model.tflite")

print(
    "Original TFLite Model Size:",
    original_size / 1024,
    "KB"
)
```

```
Original TFLite Model Size: 400.10546875 KB
```

9. Create Original TFLite Interpreter

```
original_interpreter = tf.lite.Interpreter(
    model_path="original_model.tflite"
)
```

```
original_interpreter.allocate_tensors()

original_input = original_interpreter.get_input_details()
original_output = original_interpreter.get_output_details()

print("Input:")
print(original_input)

print("\nOutput:")
print(original_output)
```

```
Input:
[{'name': 'serving_default_keras_tensor:0', 'index': 0, 'shape': array(

Output:
[{'name': 'StatefulPartitionedCall_1:0', 'index': 14, 'shape': array([
/usr/local/lib/python3.13/dist-packages/tensorflow/lite/python/interpre
    TF 2.20. Please use the LiteRT interpreter from the ai_edge_litert
    See the [migration guide](https://ai.google.dev/edge/litert/migrati
    for details.

  warnings.warn(_INTERPRETER_DELETION_WARNING)
```

10. Original Prediction

```
sample = x_test[0:1]

original_interpreter.set_tensor(
    original_input[0]["index"],
    sample
)

original_interpreter.invoke()

output = original_interpreter.get_tensor(
    original_output[0]["index"]
)

original_prediction = np.argmax(output)

print("Actual Label:", y_test[0])
print("Original Prediction:", original_prediction)
```

```
Actual Label: 7
Original Prediction: 7
```

11. Original Inference Latency

```
num_runs = 100

start = time.perf_counter()

for _ in range(num_runs):
    original_interpreter.set_tensor(
        original_input[0]["index"],
        sample
    )
    original_interpreter.invoke()

end = time.perf_counter()

original_latency = (
    (end - start) / num_runs
) * 1000

print("Original Average Latency:", original_latency, "ms")
```

```
Original Average Latency: 0.03860696000003827 ms
```

12. Apply Post-Training Quantization

```
converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [
    tf.lite.Optimize.DEFAULT
]

quantized_tflite = converter.convert()

with open("quantized_model.tflite", "wb") as f:
    f.write(quantized_tflite)

print("Quantized model saved successfully.")
```

```
Saved artifact at '/tmp/tmpb8gmc8w9'. The following endpoints are avail

* Endpoint 'serve'
  args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 28, 28), dtype=tf.f
Output Type:
  TensorSpec(shape=(None, 10), dtype=tf.float32, name=None)
Captures:
  132181235188560: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132178791675664: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132178791675280: TensorSpec(shape=(), dtype=tf.resource, name=None)
  132178791676048: TensorSpec(shape=(), dtype=tf.resource, name=None)
Quantized model saved successfully.
```

13. Measure Quantized Model Size

```
quantized_size = os.path.getsize("quantized_model.tflite")

print(
    "Quantized Model Size:",
    quantized_size / 1024,
    "KB"
)

size_reduction = (
    (original_size - quantized_size)
    / original_size
) * 100

print("Model Size Reduction:", size_reduction, "%")
```

```
Quantized Model Size: 104.078125 KB
Model Size Reduction: 73.98732756011599 %
```

14. Load Quantized Model

```
quantized_interpreter = tf.lite.Interpreter(
    model_path="quantized_model.tflite"
)

quantized_interpreter.allocate_tensors()

quantized_input = quantized_interpreter.get_input_details()
quantized_output = quantized_interpreter.get_output_details()

print("Input:")
print(quantized_input)

print("\nOutput:")
print(quantized_output)
```

```
Input:
[{'name': 'serving_default_keras_tensor:0', 'index': 0, 'shape': array(

Output:
[{'name': 'StatefulPartitionedCall_1:0', 'index': 14, 'shape': array([
/usr/local/lib/python3.13/dist-packages/tensorflow/lite/python/interpre
    TF 2.20. Please use the LiteRT interpreter from the ai_edge_litert
    See the [migration guide](https://ai.google.dev/edge/litert/migrati
    for details.
```

```
  warnings.warn(_INTERPRETER_DELETION_WARNING)
```

15. Quantized Prediction

```
quantized_interpreter.set_tensor(
    quantized_input[0]["index"],
    sample
)

quantized_interpreter.invoke()

output = quantized_interpreter.get_tensor(
    quantized_output[0]["index"]
)

quantized_prediction = np.argmax(output)

print("Actual Label:", y_test[0])
print("Quantized Prediction:", quantized_prediction)
```

```
Actual Label: 7
Quantized Prediction: 7
```

16. Quantized Inference Latency

```
start = time.perf_counter()

for _ in range(num_runs):
    quantized_interpreter.set_tensor(
        quantized_input[0]["index"],
        sample
    )
    quantized_interpreter.invoke()

end = time.perf_counter()

quantized_latency = (
    (end - start) / num_runs
) * 1000

print("Quantized Average Latency:", quantized_latency, "ms")
```

```
Quantized Average Latency: 0.04453158999979223 ms
```

17. Quantized Model Accuracy

```
correct = 0

for i in range(len(x_test)):
    input_data = x_test[i:i+1]

    quantized_interpreter.set_tensor(
        quantized_input[0]["index"],
        input_data
    )

    quantized_interpreter.invoke()

    output = quantized_interpreter.get_tensor(
        quantized_output[0]["index"]
    )

    prediction = np.argmax(output)

    if prediction == y_test[i]:
        correct += 1

quantized_accuracy = correct / len(x_test)

print(
    "Quantized Model Accuracy:",
    quantized_accuracy * 100,
    "%"
)
```

```
Quantized Model Accuracy: 97.39 %
```

18. Calculate Accuracy Difference

```
accuracy_difference = (
    original_accuracy - quantized_accuracy
) * 100

print(
    "Accuracy Difference:",
    accuracy_difference,
    "percentage points"
)
```

```
Accuracy Difference: 2.01225280882511e-06 percentage points
```

19. Calculate Latency Change

19. Calculate Latency Change

```
latency_change = (
    (original_latency - quantized_latency)
    / original_latency
) * 100

print("Latency Change:", latency_change, "%")
```

```
Latency Change: -15.34601532922584 %
```

20. Final Model Comparison

```
print("\n========================================")
print(" MODEL OPTIMIZATION RESULTS")
print("========================================")

print(
    "\nOriginal Model Size:",
    original_size / 1024,
    "KB"
)

print(
    "Quantized Model Size:",
    quantized_size / 1024,
    "KB"
)

print(
    "Size Reduction:",
    size_reduction,
    "%"
)

print(
    "\nOriginal Accuracy:",
    original_accuracy * 100,
    "%"
)

print(
    "Quantized Accuracy:",
    quantized_accuracy * 100,
    "%"
)

print(
```

```
print(
    "Accuracy Difference:",
    accuracy_difference,
    "percentage points"
)

print(
    "\nOriginal Latency:",
    original_latency,
    "ms"
)

print(
    "Quantized Latency:",
    quantized_latency,
    "ms"
)

print(
    "Latency Change:",
    latency_change,
    "%"
)

print("\n========================================")
```

```
========================================
 MODEL OPTIMIZATION RESULTS
========================================

Original Model Size: 400.10546875 KB
Quantized Model Size: 104.078125 KB
Size Reduction: 73.98732756011599 %

Original Accuracy: 97.39000201225281 %
Quantized Accuracy: 97.39 %
Accuracy Difference: 2.01225280882511e-06 percentage points

Original Latency: 0.03860696000003827 ms
Quantized Latency: 0.04453158999979223 ms
Latency Change: -15.34601532922584 %

========================================
```